SingleFlight Implementation Design for im-cds

Overview

SingleFlight is a request deduplication mechanism that ensures only ONE backend call is made for concurrent requests to the same key. Other goroutines wait and share the result.

Problem Statement

Current Behavior (Without SingleFlight)

When multiple concurrent requests come for the same entity (e.g., `GROUP#123`):

```
Time 0ms:  Request 1 for GROUP#123 → DDB call starts
Time 1ms:  Request 2 for GROUP#123 → DDB call starts (DUPLICATE!)
Time 2ms:  Request 3 for GROUP#123 → DDB call starts (DUPLICATE!)
Time 50ms: All 3 DDB calls complete

Result: 3 DDB calls, 3x DDB throughput consumed
```

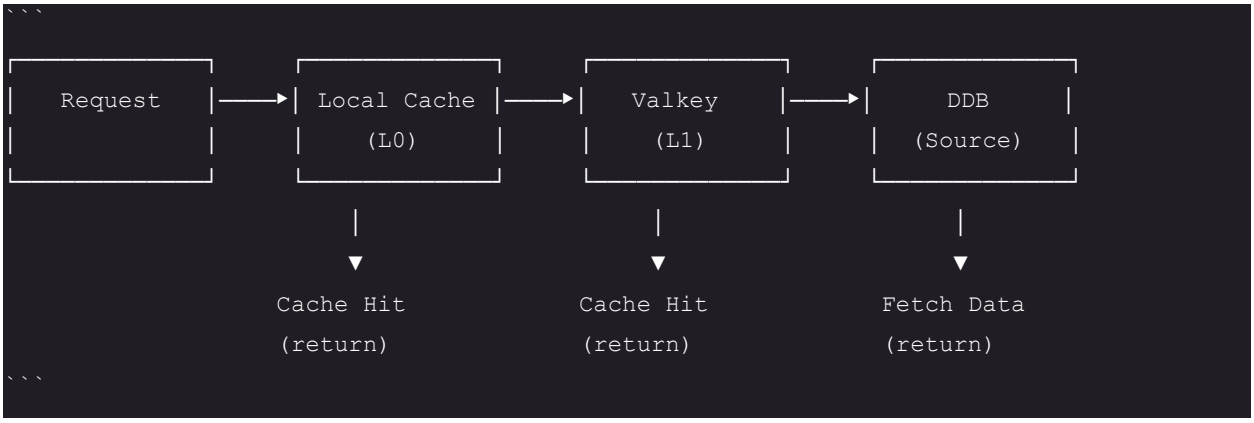
With SingleFlight

```
Time 0ms:  Request 1 for GROUP#123 → DDB call starts
Time 1ms:  Request 2 for GROUP#123 → waits for Request 1
Time 2ms:  Request 3 for GROUP#123 → waits for Request 1
Time 50ms: DDB call completes, all 3 requests get result

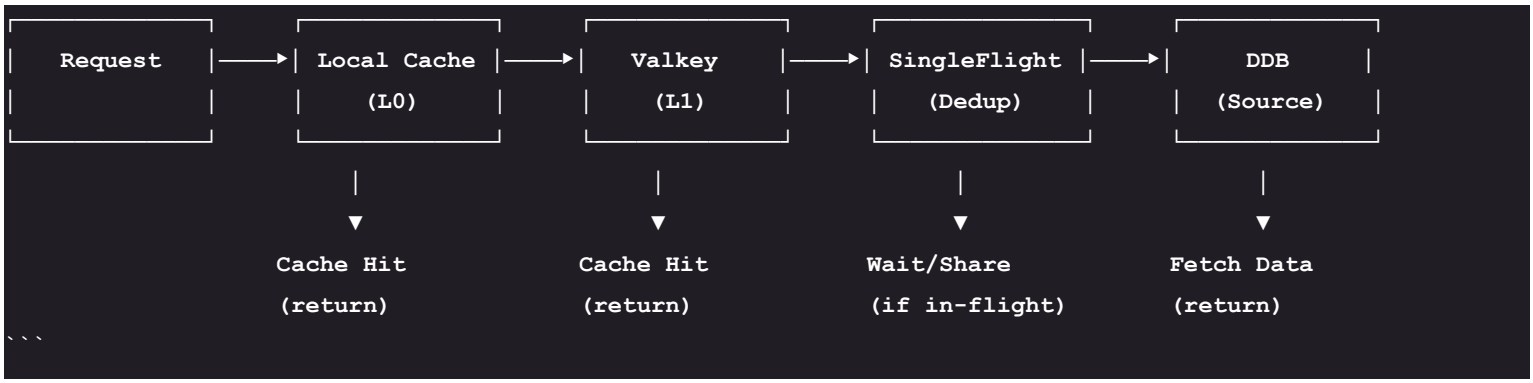
Result: 1 DDB call, ~66% throughput saved
```

Architecture

Current Data Flow



Proposed Data Flow with SingleFlight



Key Point:** SingleFlight is added ONLY at the DDB layer, NOT at cache layers.

Why Only at DDB Layer?

Layer	Deduplication Needed?	Reason
Local Cache	No	In-memory, microsecond latency, no benefit
Valkey	No	Already handles concurrent reads efficiently
DDB	Yes	High latency (50-100ms), throughput costs money

Components Affected

1. Grouper DAL (`im-cds/internal/pipeline/external/grouper/instamart/dal.go`)

Functions to wrap with SingleFlight: (2-3 more func)
- `batchGetGroupsFromDDB()` - fetches groups after Valkey miss
- `fetchGroupDetailsFromDDB()` - fetches group details after Valkey miss
- `batchGetGroupInfoFromDDB()` - fetches group info after Valkey miss

2. Margin DAL (`im-cds/internal/pipeline/external/marginer/dal.go`)

Functions to wrap with SingleFlight:
- `getMarginFromDDB()` - fetches margins after Valkey miss

3. Anchor Price DAL (`im-cds/internal/pipeline/external/anchor/dal.go`)

Functions to wrap with SingleFlight:
- `getAnchorPriceFromDDB()` - fetches anchor prices after Valkey miss

Implementation Plan

Phase 1: Core SingleFlight Package

Location: `im-cds/internal/utils/singleflight/`

```
Go
// singleflight.go - Core wrapper around
golang.org/x/sync/singleflight
type SingleFlightGroup struct {
    group   singleflight.Group
    name    string // for metrics
    enabled bool
}

func (g *SingleFlightGroup) Do(ctx context.Context, key string, fn
func() (interface{}, error)) Result
```

Phase 2: Configuration

Location: `im-cds/config/config.go`

```Go
// Add to Config struct
SingleFlightEnabled        bool `env:"SINGLEFLIGHT_ENABLED"
envDefault:"false"`
SingleFlightGrouperEnabled bool `env:"SINGLEFLIGHT_GROUPER_ENABLED"
envDefault:"false"`
SingleFlightMarginEnabled  bool `env:"SINGLEFLIGHT_MARGIN_ENABLED"
envDefault:"false"`
SingleFlightAnchorEnabled  bool `env:"SINGLEFLIGHT_ANCHOR_ENABLED"
envDefault:"false"`
```

Location: `im-cds/cmd/cdsserver/conf/integ.conf` and `prod.conf`

```None
# SingleFlight Configuration
SINGLEFLIGHT_ENABLED=false
SINGLEFLIGHT_GROUPER_ENABLED=false
SINGLEFLIGHT_MARGIN_ENABLED=false
SINGLEFLIGHT_ANCHOR_ENABLED=false
```

Phase 3: Metrics

Location: `im-cds/internal/cds/constants/constants.go`

```Go
// SingleFlight metrics
SingleFlightGrouperTotal   = "singleflight_grouper_total"
SingleFlightGrouperShared  = "singleflight_grouper_shared"
SingleFlightMarginTotal    = "singleflight_margin_total"
SingleFlightMarginShared   = "singleflight_margin_shared"
SingleFlightAnchorTotal    = "singleflight_anchor_total"
```

```
SingleFlightAnchorShared   = "singleflight_anchor_shared"
```

Phase 4: DAL Integration

Grouper DAL Changes example

```Go
// Add to Dao struct
type Dao struct {
    Config         *config.Config
    Provider       store.Provider
    ProviderNonDAX store.Provider
    CacheService   CacheServiceInterface
    singleFlight   *singleflight.SingleFlightGroup  // NEW
}

// Modified batchGetGroupsFromDDB
func (d *Dao) batchGetGroupsFromDDB(ctx context.Context, entityIds
[]string) ([]*store.BatchGetOutput, error) {
    if d.singleFlight == nil || !d.singleFlight.IsEnabled() {
        return d.batchGetGroupsFromDDBDirect(ctx, entityIds)
    }

    // Use singleflight for each key
    results := make([]*store.BatchGetOutput, 0, len(entityIds))
    var wg sync.WaitGroup
    var mu sync.Mutex

    for _, entityId := range entityIds {
        wg.Add(1)
        go func(id string) {
            defer wg.Done()

            result := d.singleFlight.Do(ctx, id, func()
(interface{}, error) {
```

```
                return d.fetchSingleEntityFromDDB(ctx, id)
            })

            mu.Lock()
            if result.Val != nil {
                results = append(results,
result.Val.(*store.BatchGetOutput))
            }
            if result.Shared {
                metrics.IncrementCounterWithBlLabels(ctx,
constants.SingleFlightGrouperShared)
            }
            mu.Unlock()
        }(entityId)
    }

    wg.Wait()
    return results, nil
}
```

FAQ

Q: Why not use singleflight at Valkey layer?

A: Valkey is already fast (1-5ms) and handles concurrent reads efficiently. The overhead of singleflight coordination would negate any benefits.

Q: What if the first request fails?

A: All waiting requests will receive the same error. This is correct behavior - if DDB is down, all requests should fail.

Q: Does this affect cache invalidation?

A: No. SingleFlight only affects in-flight requests. Once a request completes, the key is removed from singleflight tracking.

Q: What about different request contexts?

A: All requests share the same result regardless of context. This is safe because we're only deduplicating DDB reads, which return the same data for the same key.